

Evolving with AI: A Longitudinal Analysis of Developer Logs

Nikhil Anand

April 1, 2026

The rapid integration of AI-powered coding assistants into modern development environments has transformed software engineering into a form of human-AI collaboration. While tools such as GitHub Copilot and JetBrains AI Assistant promise improvements in productivity and efficiency, their long-term impact on real-world developer workflows remains insufficiently understood. Most prior research has relied on short-term experiments or self-reported perceptions, leaving a gap in understanding how developers' behaviors evolve over time when AI becomes embedded in everyday programming practices.

To address this gap, the paper presents a mixed-method longitudinal study combining telemetry data from 800 developers over two years with survey and interview insights. By analyzing five key workflow dimensions—productivity, code quality, code editing, code reuse, and context switching—the study offers a comprehensive view of how AI reshapes development practices. Importantly, it contrasts what developers actually do (behavioral logs) with what they perceive (survey responses), revealing nuanced differences between observed and perceived impacts.

1 Motivation

The primary motivation of this work arises from the gap between widespread adoption of AI coding assistants and the limited understanding of their long-term effects on developer workflows. While tools such as GitHub Copilot and similar systems are increasingly embedded in IDEs, most prior studies focus on short-term experiments or subjective perceptions rather than sustained behavioral change. As a result, there is a lack of empirical evidence on how developer practices evolve over time with continuous AI usage [1, 2, 3].

Another key motivation lies in the limitations of existing research methodologies, which often rely on controlled lab studies or self-reported data. These approaches fail to capture the complexity of real-world development and may not generalize to practical settings. Moreover, prior work shows that developer perceptions frequently diverge from actual behavior, making it necessary to complement subjective reports with objective telemetry data [4].

The study is further motivated by the need to understand how AI affects multiple dimensions of development workflows beyond productivity, including code quality, editing effort, reuse, and context switching. Prior research presents conflicting findings on productivity gains, with some studies reporting significant improvements while others highlight negligible or even negative effects, indicating a fragmented understanding of AI's real impact [5, 6, 7].

Finally, the authors aim to address the lack of longitudinal, real-world evidence by leveraging large-scale telemetry data. By combining behavioral logs with survey data, the study seeks to uncover how AI reshapes developer workflows in subtle and often unrecognized ways, providing insights that can guide the design of future AI-assisted development tools and improve evaluation methodologies in human-AI collaboration research.

2 Methodology

The study employs a mixed-method approach combining quantitative telemetry analysis with qualitative survey and interview data. This dual perspective allows the authors to examine both observable developer

behavior and subjective perceptions, ensuring a more comprehensive understanding of AI’s impact on workflows.

The first component consists of a survey of 62 professional developers, complemented by follow-up interviews. The survey includes Likert-scale questions addressing changes in productivity, code quality, editing, reuse, and context switching. Additional open-ended responses and interviews provide deeper insights into developers’ reasoning and experiences, capturing contextual and qualitative nuances.

The second component involves a large-scale longitudinal analysis of IDE telemetry logs, comprising over 151 million interaction events from 800 developers (400 AI users and 400 non-users). The data spans two years and includes fine-grained actions such as typing, deletions, debugging, pasting external content, and IDE switching. These actions are used as proxies for key workflow dimensions, enabling systematic behavioral analysis.

For analysis, the authors aggregate data monthly and apply mixed-effects linear regression models to identify trends over time and differences between AI users and non-users. Statistical tests account for non-normal distributions and variability across users, ensuring that observed trends are robust and statistically significant.

3 Key Findings

The study finds that AI users exhibit significantly higher levels of code production, as evidenced by a steep increase in typed characters compared to non-users. At the same time, they also show a substantial increase in code deletions, indicating that AI-assisted development involves more iterative refinement rather than straightforward efficiency gains.

In terms of code quality, results are mixed. While developers perceive slight improvements or no change, telemetry data shows stable debugging activity for AI users, contrasting with a decline among non-users. This suggests that AI may not necessarily reduce errors but instead shifts how and when issues are addressed during development.

The findings also reveal that code reuse and context switching increase over time for AI users, albeit modestly. Developers rely more on external sources and AI-generated snippets, while also experiencing greater workflow fragmentation, as indicated by increased context switching events. This challenges the assumption that AI reduces interruptions in development workflows.

A particularly important insight is the discrepancy between perception and behavior. While developers report clear productivity gains, they often perceive little change in other aspects of their workflow. However, telemetry data shows significant structural changes in coding practices, highlighting that AI’s impact is often subtle and not fully recognized by users.

4 Implications

The study highlights that AI tools should not be viewed solely as productivity enhancers but as systems that reshape the structure and rhythm of development work. This suggests a need to rethink how productivity is measured, moving beyond output metrics to consider effort distribution, cognitive load, and workflow fragmentation.

For tool designers, the findings emphasize the importance of supporting continuity and reducing cognitive overhead. Since AI increases editing and context switching, future tools should aim to minimize fragmentation and provide better integration of suggestions within developer workflows.

Another implication is the need for greater transparency and awareness in AI-assisted development. Developers often underestimate how their workflows change, indicating that tools should provide feedback on usage patterns, editing behavior, and reliance on AI-generated code to support informed decision-making.

Finally, the study underscores the importance of combining behavioral and perceptual data in evaluating AI systems. Relying solely on self-reports can lead to incomplete conclusions, whereas integrating teleme-

try provides a more accurate picture of real-world impact. This has broader implications for research methodologies in human–AI interaction.

5 Threats to Validity

One key threat to validity arises from construct validity, as the study relies on proxies (e.g., typed characters for productivity, debugging for quality). While these measures are grounded in prior work, they may not fully capture the complexity of developer activities or intent. Additionally, survey responses and telemetry capture different constructs, making direct comparisons challenging.

Internal validity is limited by self-selection bias and lack of experimental control. AI users may inherently differ from non-users in motivation, experience, or work habits, which could influence observed trends. As a result, the study identifies associations rather than causal effects, and differences between groups should be interpreted cautiously.

External validity is another concern, as the dataset is derived from JetBrains IDE users and a specific AI assistant. While the study includes diverse languages and environments, the findings may not generalize fully to other tools or development contexts. Nevertheless, the core workflow dimensions analyzed are broadly applicable, supporting partial generalizability of the results.

References

- [1] Saki Imai. “Is GitHub Copilot a substitute for human pair-programming?” In: *ICSE Companion*. 2022.
- [2] Priyan Vaithilingam, Tianyi Zhang, and Elena Glassman. “Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models”. In: *CHI Extended Abstracts*. 2022.
- [3] Jenny T Liang, Chenyang Yang, and Brad A Myers. “A large-scale survey on the usability of AI programming assistants: Successes and challenges”. In: *ICSE*. 2024.
- [4] Prem Devanbu, Thomas Zimmermann, and Christian Bird. “Belief & evidence in empirical software engineering”. In: *ICSE*. 2016.
- [5] Sida Peng et al. “The impact of AI on developer productivity: Evidence from GitHub Copilot”. In: *arXiv preprint arXiv:2302.06590* (2023).
- [6] Albert Ziegler et al. “Productivity assessment of neural code completion”. In: *SIGPLAN Machine Programming*. 2022.
- [7] Joel Becker et al. “Measuring the impact of early-2025 AI on experienced open-source developer productivity”. In: *arXiv preprint arXiv:2507.09089* (2025).